Mentimeter

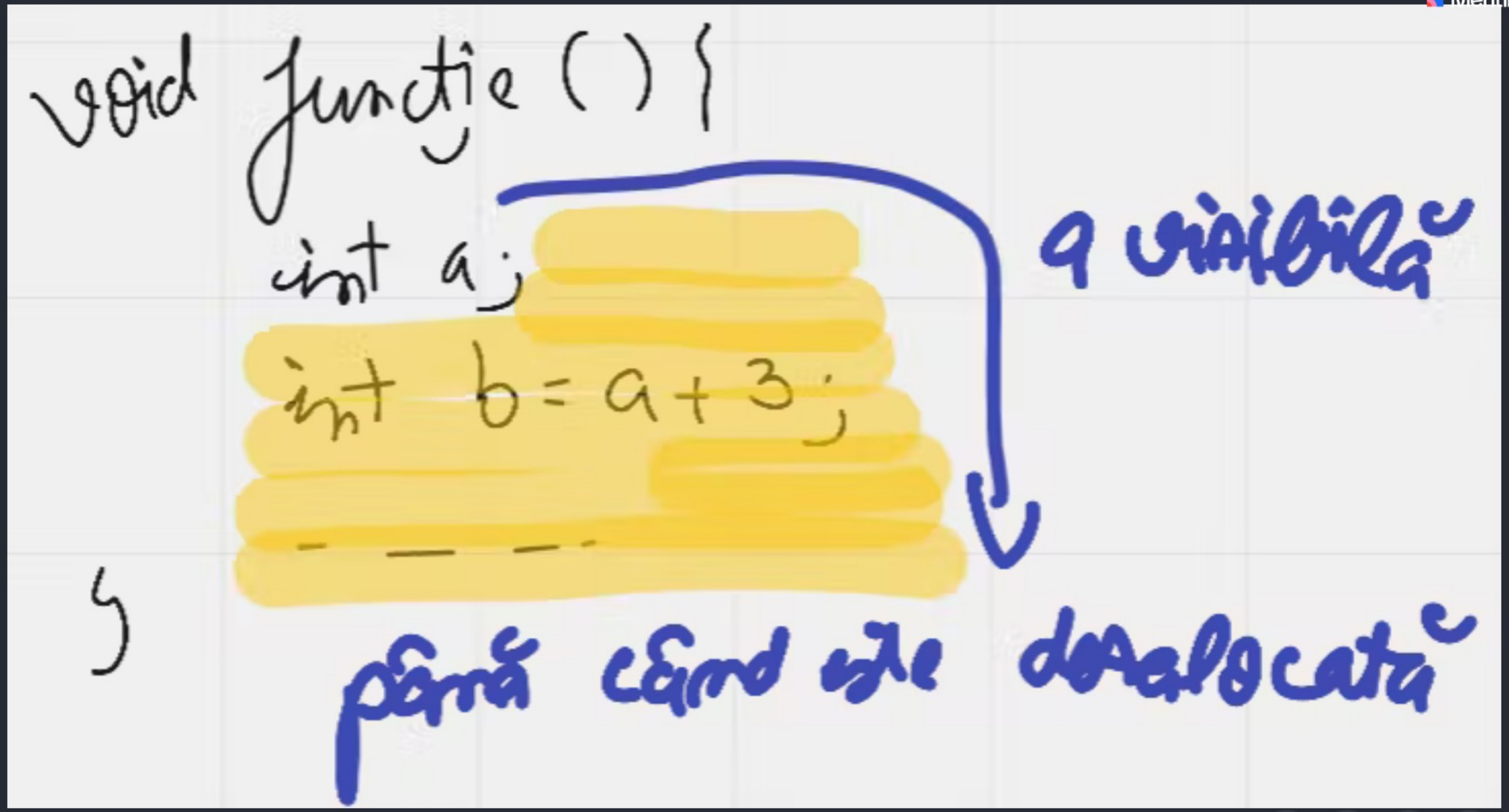
Racket: Legarea variabilelor. Închideri funcționale

Mentimeter

Domeniul de vizibilitate al unei variabile x

porțiunile din program în care x
poate fi accesat (**este vizibil**)

Mentimeter


void functie ( ) {
int a;
int b = a + 3;
_ _ _ _
}
a variabila
poate exista sau nu dealocata

Mentimeter

Context computațional al unui program P la un moment t

mulțimea variabilelor vizibile (+valorile lor) în punctul P la momentul t

Mentimeter

```
int main () {
    int a = 1;
    char c = 'd';
    Student s = new Student(..);
}
```

CONTEXT

a : 1

c : 'd'

s : new Student(..)

Mentimeter

Legare (name binding)

- pereche de identificator + valoare
- statica (la compilare) - context nou la fiecare legare
- dinamica (la rulare - ex: dispatch) - se cauta identificatorii in contextul global

Mentimeter

Binding vs assignment

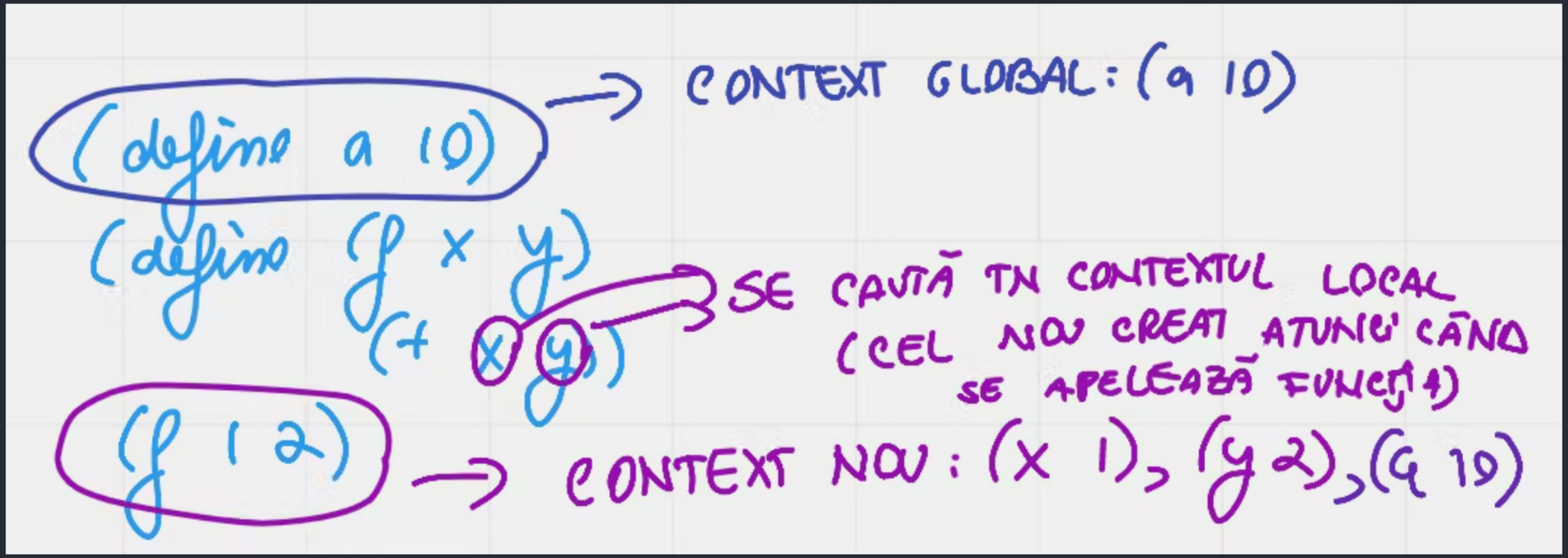
- Binding: se initializeaza variabila in context
- Assignment: se atribuie o valoare variabilei

Mentimeter

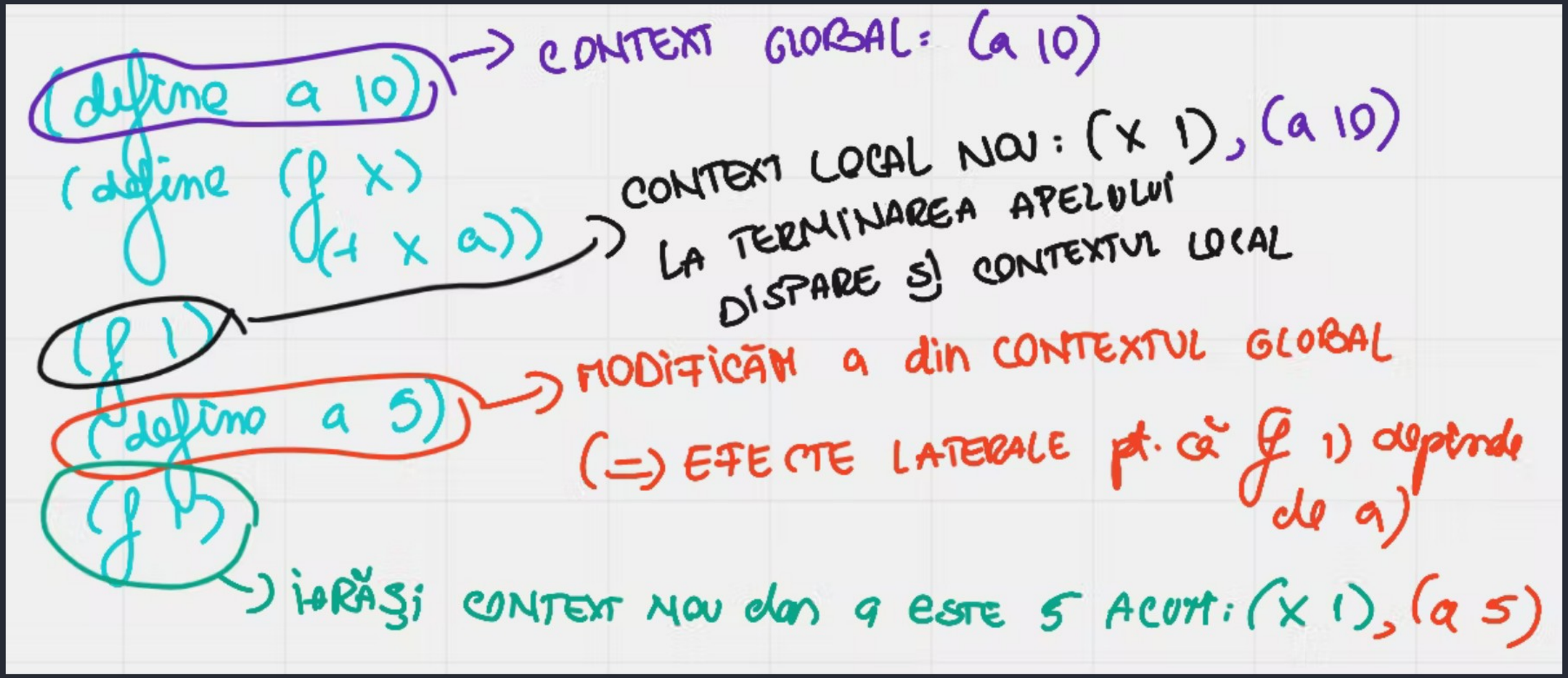
let

creeaza variabile vizibile numai in corpul let-ului

Mentimeter



Legarea statica



Legarea dinamica

Mentimeter

(let (declaratii (id valoare)) corp-let)

- variabilele definite in let **sunt vizibile de-abia in corpul let-ului**
- daca definim a si vrem ca b sa foloseasca valoarea anterioara a lui a atunci ne trebuie alt tip de let
- la let normal b o sa ia valoarea lui a din contextul global

Mentimeter

```
; (let ((a 1) (b (+ a 2))) (+ a b)) ; eroare b nu vede a-ul
                                    ; a devine vizibil de-abia in corpul lui let (+ a b)

(define a 10)
(let ((a 1) (b (+ a 1))) (+ a b)) ; b vede a-ul definit global => b = 11
```

Exemplu let



Mentimeter

Vrem ca b sa vada valoarea lui a din let - solutii?

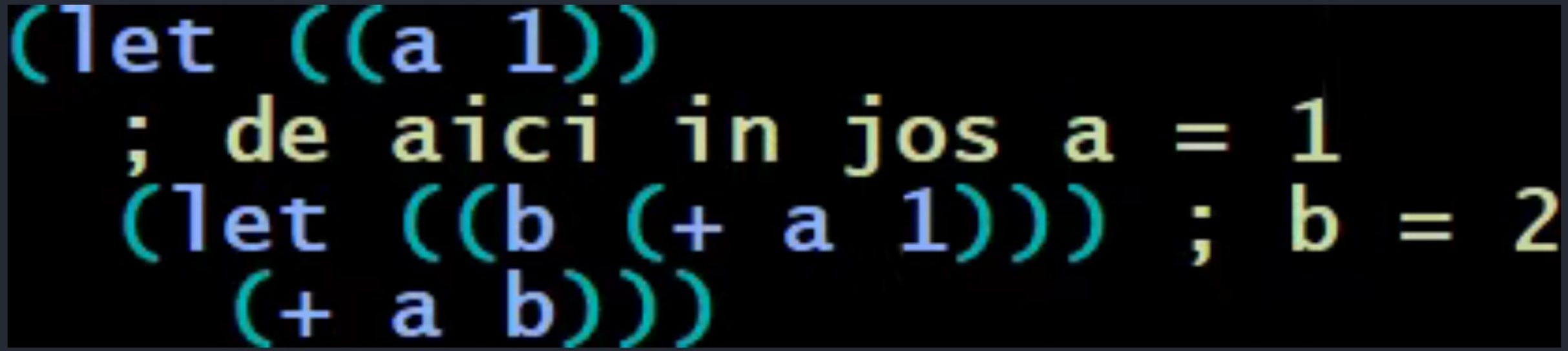
- let* (incoming)
- let-uri imbricate

Mentimeter

Let-uri imbricate

- separam variabilele in let-uri diferite
- primul let defineste variabila a
- in corpul primului let definim al doilea let pentru b (**o sa vada valoarea lui a**)

Mentimeter



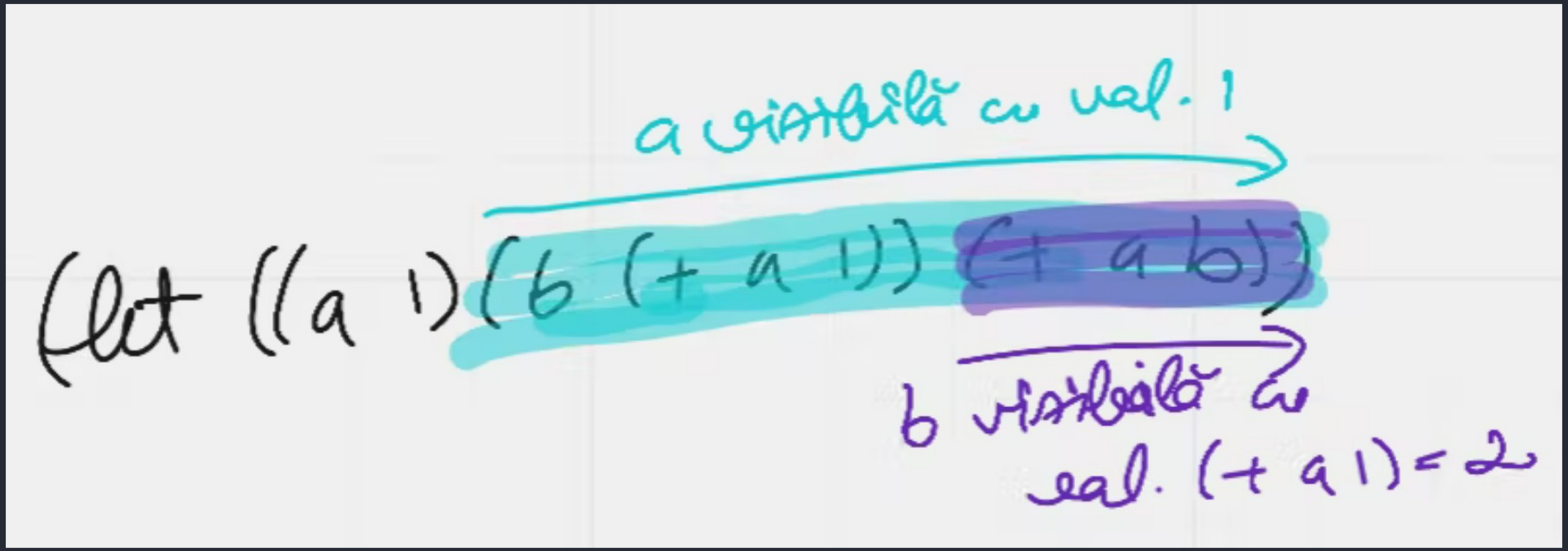
Solutie

Mentimeter

let*

- similar let
- daar aici variabilele devin vizibile imediat dupa definire

Mentimeter



Exemplu

Mentimeter

letrec

- variabilele sunt vizibile si inainte de definirea in let
- **daar atunci cand sunt folosite trebuie sa le stim valoarea**
- avem eroare daca trebuie sa evaluam o expresie care contine o variabila inca nedefinita

Mentimeter

```
;(letrec ((a (+ b 1)) (b 2)) ; la definirea lui a se face evaluarea lui (+ b 1)
;   (+ a b))                 ; nu stim inca b => eroare

(letrec ((f (λ(b)(+ b 1))) (b 2) (a (f b))) ; f este o functie care inca nu se apeleaza
  (+ a b))                                  ; cand ajungem la a stim deja cine e b
```

Exemple

Mentimeter

Named let

- numele dat = procedura
- putem sa o apelam recursiv dandu-i ca parametri variabilele din lista de definiri

Mentimeter

```
(define (interval n)
  (let loop ((start 1) (stop n) (numbers '()))
    (if (> start stop)
        (reverse numbers)
        (loop (+ start 1) stop (cons start numbers)))))
(interval 10)
```

Exemplu

Mentimeter

(let-values (((id1 id2 ...) expr)) corp)

- avem expresii care intorc mai multe valori
- ex: quotient/remainder a b intoarce si catul si restul
- initializam mai multe variabile odata

Mentimeter

```
(range 1 4)                    ; numerele de la 1 la 3
(apply values (range 1 4)); values intoarce cate un element din lista separat
                           ; (tipul intors: multiple-values)

(let-values (((a b c) (apply values (range 1 4)))) ; a = 1, b = 2, c = 3
  (print a)                                        ; fiecare variabila ia pe rand
  (print b)                                        ; cate o valoare din cele intoarse
  (print c))                                       ; si dupa se afiseaza variabilele
```

Exemplu



Mentimeter

let*-values

- let-values cu avantajul de la let*
- variabilele sunt vizibile odata ce le definim

Mentimeter

Atentie

trebuie date atatea variabile cate valori sunt intoarse (altfel tipa arity mismatch)

Mentimeter

Inchideri functionale

- la ce se evalueaza o lambda-expresie?
- textul ei + contextul computational

Mentimeter

De ce?

- contextul computational contine variabilele referite de expresie dar definite in afara ei
- cand aplicam functia evaluam si variabilele (valorile lor se pot schimba intre timp)
- **pot fi folosite pentru intarzierea evaluarii**

Mentimeter

(splitf-at L f)

- intoarce 2 liste
- sparge lista la prima valoare pe care f intoarce #f
- prima lista = toate elementele pana ce f intoarce #f
- a doua lista = de la prima valoare pe care f intoarce #f incolo

Mentimeter

Exemplu splitf-at

- (splitf-at '(1 2 3 -2 3 0 -4) (lambda(x) (> x 0)))
- ex: '(1 2 3), '(-2 3 0 -4)